SecureNova AI Identity Security â€” Walkthrough for the CTO

This is the plain-language version of the capstone findings: what was tested, what we found, and what to do about it. No security jargon â€ if a term is unavoidable, it's explained the first time it's used.

Scope and Methodology, in Plain Terms

SecureNova's AI customer-service assistant doesn't just talk to customers â€ it also calls internal systems, looks things up in a knowledge base, and can take actions like issuing refunds, all on its own. Each of those connections is a place where something could go wrong if the wrong person (or the wrong instruction) gets in.

We did four things, in order:

1. **Mapped every "identity"** the assistant and its systems use â€ not just human logins, but the AI agent itself, its API keys, and the automated services it talks to â€ and wrote down what each one is allowed to do.
2. **Built the real login and permission system** (via Auth0, an industry-standard identity provider) the way it should work in production: short-lived access, multi-factor login for staff, and the AI assistant only ever acting within the permissions of the person it's helping.
3. **Attacked our own system** the way a real attacker would â€ specifically, by hiding instructions inside documents the assistant reads, and by trying to trick one internal AI component into following orders that claimed to come from another, trusted one.
4. **Built and tested defenses** against everything the attacks found: filters that catch suspicious instructions before the assistant acts on them, digital signatures so internal components can prove who they really are, and automated alerts that flag unusual activity.

Everything below is built on real, reproducible tests â€ not guesswork.

Risk Priority Matrix

Plain-language severity, in order of what to fix first. ("CVSS score" is an industry-standard 0-10 severity rating for a security issue; higher means worse.)

Priority	What could go wrong	How bad, in business terms	Severity score
1	A hidden instruction inside a document the assistant reads tricks it into leaking an internal credential	A leaked key could let someone rack up usage costs on our account, or be a stepping stone to a bigger breach	*[fill in from CVSS-Findings-Table.xlsx, RT-01]*
2	A message pretending to be from a trusted internal system convinces another AI component to take an action (like issuing a refund) it shouldn't	Real financial loss or incorrect customer actions, with no human ever approving it	*[fill in, RT-02]*
3	Asking the assistant the right way reveals its internal instructions	Mostly a competitive/privacy concern â€" less about data loss, more about understanding how to attack the system further	*[fill in, RT-03]*
4	A document in the knowledge base is written to make the assistant take an action on its own	Same financial/data risk as #2, but reached through content rather than a direct message	*[fill in, RT-04/RT-05]*

Fill in the severity scores from your completed `03-red-team/CVSS-Findings-Table.xlsx` *once your red-team results are final â€" this table is ready to receive them. Order the rows by actual score once you have them; this draft assumes items 1 and 2 will score highest, which is typical for this attack pattern, but verify against your real numbers.*

Top 5 Recommendations

#	Recommendation	Effort	Business rationale
1	Require a human to approve any AI-initiated refund or data export above a set value	Low (1-2 weeks) ” this is a workflow change, not new infrastructure	Closes the highest-impact risk directly: even if every other defense fails, nothing financially damaging happens without a person signing off
2	Keep the filters that catch suspicious instructions (deployed in this project) running in production, and review what they catch weekly	Low (ongoing, ~1 hour/week)	Already built and tested; the cost here is just keeping it maintained, which is far cheaper than the cost of a single successful attack
3	Make sure internal AI components only trust messages that are digitally signed, not just labelled with a name	Medium (2-4 weeks to extend beyond this project's scope to all internal components)	Removes an entire class of attack (impersonation) rather than just filtering its symptoms
4	Shorten how long credentials and login sessions stay valid, and auto-rotate them	Low (already configured in this project; needs confirming in production)	Limits how much damage a leaked credential can do, since it expires quickly even if nothing else catches the leak
5	Set up automated alerts for unusual AI activity (sudden spikes in requests, a component acting outside its normal pattern) and route them to a real person	Medium (2-3 weeks to connect to an on-call process)	Turns a silent breach into a noticed one within minutes instead of days ” most of the cost of a breach comes from how long it goes unnoticed

What "done" looks like

All five of these are either fully built and tested in this project, or a short, well-scoped next step from something that already is. None require a ground-up redesign of the assistant.